A. Regressor model

Servo table dynamics

$\tau_t \ddot{\theta} + \dot{\theta} = \bar{K}_t u(t) - \bar{K}_t A_c \cdot \mathrm{sign}(\dot{\theta}) - \bar{K}_t \bar{K}_a \ddot{\alpha}_1$

Regressor model $y_1 = \varphi_1^T \beta_1$

where:

the output $y_1 = u$

the regressor vector $\varphi_1 = \left[\ddot{\theta}, \dot{\theta}, \mathrm{sign}(\dot{\theta}), \ddot{\alpha}_1\right]^T$

the parameter vector $\beta_1 = \left[\frac{\tau_t}{\bar{K}_t}, \frac{1}{\bar{K}_t}, A_c, \bar{K}_a\right]^T$

Rod dynamics

$-\frac{L_t}{g}\omega_r^2 \ddot{\theta} = \ddot{\alpha}_1 + 2\zeta_r \omega_r \dot{\alpha}_1 + \omega_r^2 \alpha_1$

Regressor model $y_2 = \varphi_2^T \beta_2$

where:

the output $y_2 = \ddot{\alpha}_1$

the regressor vector $\varphi_2 = \left[-\ddot{\theta}, -\dot{\alpha}_1, -\alpha_1\right]^T$

the parameter vector $\beta_2 = \left[\frac{L_t}{g}\omega_r^2, 2\zeta_r \omega_r, \omega_r^2\right]^T$

B. System identification

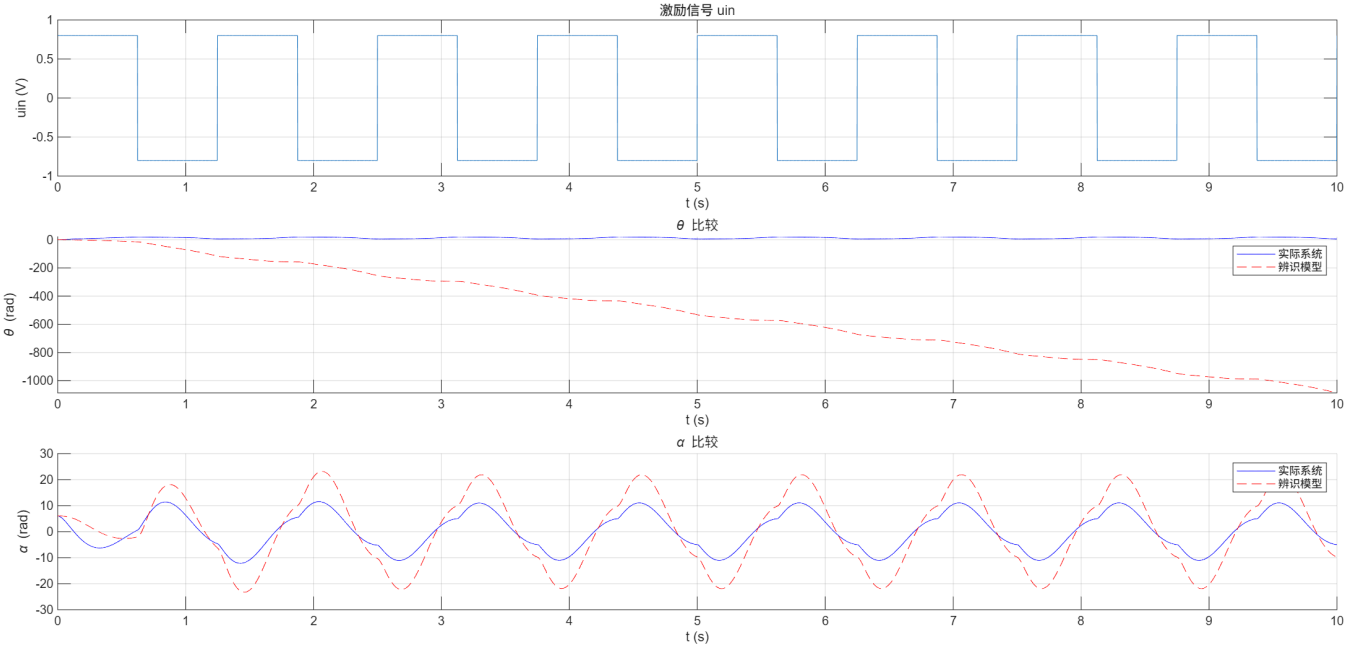
Fill in the MATLAB codes

```
四、倒立摆/单摆的系统辨识实验(辨识作业待完成)
ModelSelFlag = 3; %选择单摆模型 0: rotary table without rod, 1:nonlinear IP, 2:linear IP,3
nonlinear Pendulum, 4:linear Pendulum
IdentSet.Alpha1Flag = 0; %倒摆
%在输入信号的激励下，进行单摆系统仿真
[IPResData] =
Ident_SimulateRun(NominalPara,ModelSelFlag,UinData,InitX,DisturbPara,SimuRunInfo);
% 两边滤波处理
[FilterResData]=Ident_FilterRun(IPResData,IdentSet,SimuRunInfo,ModelSelFlag);
data = FilterResData;
t = data.time;

% --- 选择用于辨识的数据段（去掉初始瞬态） ---
tstart = 1.0; % 去掉前1s的瞬态（可根据需要调整）
tend = SimuRunInfo.TotalTime - 0.5; % 去掉最后0.5s（避免边界效应）
idx = find(t >= tstart & t <= tend);

% 读取滤波后的信号（识别时使用滤波后的量）
uin_f = data.uin_f(idx);
alpha_f = data.alpha_f(idx);
alphadot_f = data.alphadot_f(idx);
alphaddot_f = data.alphaddot_f(idx);

theta_f = data.theta_f(idx);
thetadot_f = data.thetadot_f(idx);
thetaddot_f = data.thetaddot_f(idx);

signthetadot_f = data.signthetadot_f(idx); % sign(thetadot) 渠道

% ------- 1) 旋转台（table）参数估计 -------
% 回归模型（参照推导）:
% thetaddot = (Kt/tao) * u + (-1/tao) * thetadot + (-Kc/tao) * sign(thetadot) + (-Ka/tao) *
alpha
% 令 Phi1 = [uin, thetadot, signthetadot, alpha]
Phi1 = [uin_f, thetadot_f, signthetadot_f, alpha_f];
y1 = thetaddot_f;

% 去除含 NaN 或 Inf 的行
nanmask1 = any(isnan(Phi1),2) | any(isinf(Phi1),2) | isnan(y1) | isinf(y1);
Phi1(nanmask1,:) = [];
y1(nanmask1) = [];

% 最小二乘估计
beta1 = (Phi1' * Phi1) \ (Phi1' * y1); % 4x1
% beta1 = [b1; b2; b3; b4] 对应 [Kt/tao; -1/tao; -Kc/tao; -Ka/tao]

b1 = beta1(1);
```

```
b2 = beta1(2);
b3 = beta1(3);
b4 = beta1(4);

% 由 b2 = -1/tao => tao = -1/b2
Taot_est = -1 / b2;
Kt_est = b1 * Taot_est;        % Kt = b1 * tao
Kc_est = - b3 * Taot_est;      % Kc = -b3 * tao  (Coulomb friction)
Ka_est = - b4 * Taot_est;      % Ka = -b4 * tao

% ------- 2) 杆（pendulum/rod）参数估计 -------
% 回归模型:
% alphaddot = Kr * thetaddot + (-omega_r^2) * alpha + (-2*zeta_r*omega_r) * alphadot
Phi2 = [thetaddot_f, alpha_f, alphadot_f];
y2 = alphaddot_f;

nanmask2 = any(isnan(Phi2),2) | any(isinf(Phi2),2) | isnan(y2) | isinf(y2);
Phi2(nanmask2,:) = [];
y2(nanmask2) = [];

beta2 = (Phi2' * Phi2) \ (Phi2' * y2); % [Kr; -omega^2; -2*zeta*omega]

Kr_est = beta2(1);
neg_omega2 = beta2(2);
neg_2zetaomega = beta2(3);

% 保证数值合理再开根号/求zeta
if neg_omega2 < 0
    omega_r_est = sqrt(-neg_omega2);
else
    warning('估计结果异常： beta2(2) >= 0，无法从中恢复 omega_r，设为名义值');
    omega_r_est = NominalPara.wr;
end

zeta_r_est = - neg_2zetaomega / (2 * omega_r_est);

% ------- 构造 IdentifyPara（用于后续仿真验证） -------
IdentifyPara = NominalPara; % 以名义参数为模板，覆盖被估计项
IdentifyPara.Taot = Taot_est;
IdentifyPara.Kt = Kt_est;
IdentifyPara.Ka = Ka_est;
IdentifyPara.Ac = Kc_est;      % Coulomb friction
IdentifyPara.Kr = Kr_est;
IdentifyPara.wr = omega_r_est;
IdentifyPara.kesair = zeta_r_est;
IdentifyPara.wt = 1/IdentifyPara.Taot;
IdentifyPara.deta_alpha = 0;
IdentifyPara.deta_theta = 0;

% 打印估计结果
fprintf('===== 参数估计结果 =====\n');
fprintf('Taot = %.4f  Kt = %.4f  Ka = %.4f  Ac(Kc) = %.4f\n', Taot_est, Kt_est, Ka_est,
Kc_est);
```

```
fprintf('Kr = %.4f  wr = %.4f  zeta_r = %.4f\n', Kr_est, omega_r_est, zeta_r_est);

% ------- 3) 验证：用估计参数做一次仿真并比较 -------
ModelSelFlag = ModelSelFlag; % 使用脚本中已设的 ModelSelFlag
[IPIdentVerifyData] =
Ident_SimulateRun(IdentifyPara,ModelSelFlag,UinData,InitX,DisturbPara,SimuRunInfo);

% ------- 4) 绘图比较（激励、原始与估计响应） -------
figure;
subplot(3,1,1);
plot(data.time, data.uin); grid on;
xlabel('t (s)'); ylabel('uin (V)'); title('激励信号 uin');

subplot(3,1,2);
hold on; grid on;
plot(data.time, data.theta, 'b');
plot(IPIdentVerifyData.time, IPIdentVerifyData.theta, 'r--');
xlabel('t (s)'); ylabel('\theta (rad)'); legend('实际系统','辨识模型');
title('\theta 比较');
hold off;

subplot(3,1,3);
hold on; grid on;
plot(data.time, data.alpha, 'b'); % 原始未滤 alpha
plot(IPIdentVerifyData.time, IPIdentVerifyData.alpha, 'r--');
xlabel('t (s)'); ylabel('\alpha (rad)'); legend('实际系统','辨识模型');
title('\alpha 比较');
hold off;

% ------- 5) 残差检验（以 alpha 为例） -------
% 计算实际 alphaddot 与回归模型预测的差
y2_pred = Phi2 * beta2;
% 注意 y2_pred 对应的索引是 Phi2 使用后的 idx 去除 NaN 的子集；我们显示简要残差统计
residuals = y2 - y2_pred;
fprintf('alphaddot 残差 RMS = %.6f\n', sqrt(mean(residuals.^2)));
```

Figures



Residuals

RMS(alphaddot) = 3.110181

C. Compare the system identification results

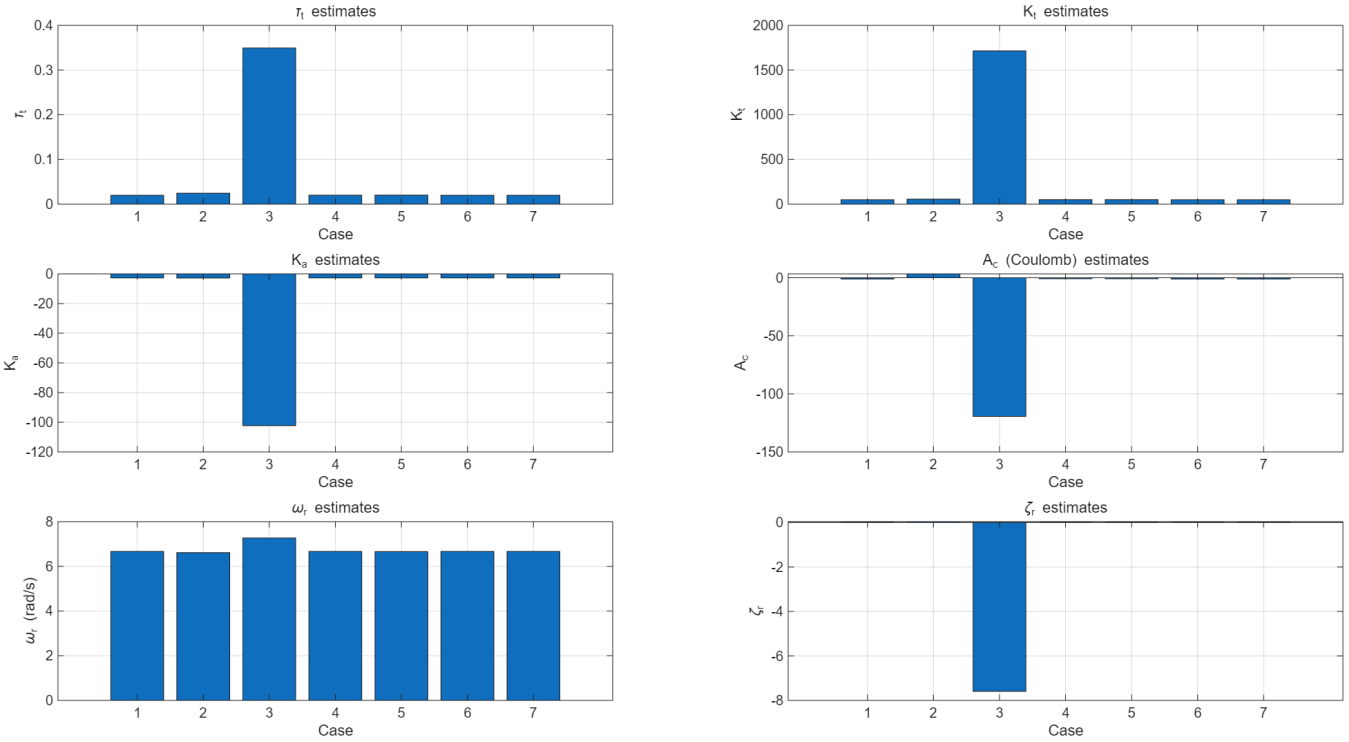
Case	Variance of measurement noise	Identical filter	Input disturbance $d_t = A_d sin(2\pi f_d t)$
Case 1	$\sigma_n = 0$	$\omega_f = 15, \zeta_f = 0.7$	$A_d = 0, f_d = 1$
Case 2	$\sigma_n = 0$	$\omega_f = 5, \zeta_f = 0.7$	$A_d = 0, f_d = 1$
Case 3	$\sigma_n = 0$	$\omega_f = 300, \zeta_f = 0.7$	$A_d = 0, f_d = 1$
Case 4	$\sigma_n = 1$	$\omega_f = 15, \zeta_f = 0.7$	$A_d = 0, f_d = 1$
Case 5	$\sigma_n = 3$	$\omega_f = 15, \zeta_f = 0.7$	$A_d = 0, f_d = 1$
Case 6	$\sigma_n = 0$	$\omega_f = 15, \zeta_f = 0.7$	$A_d = 0.1, f_d = 1$
Case 7	$\sigma_n = 0$	$\omega_f = 15, \zeta_f = 0.7$	$A_d = 0.5, f_d = 1$

Results

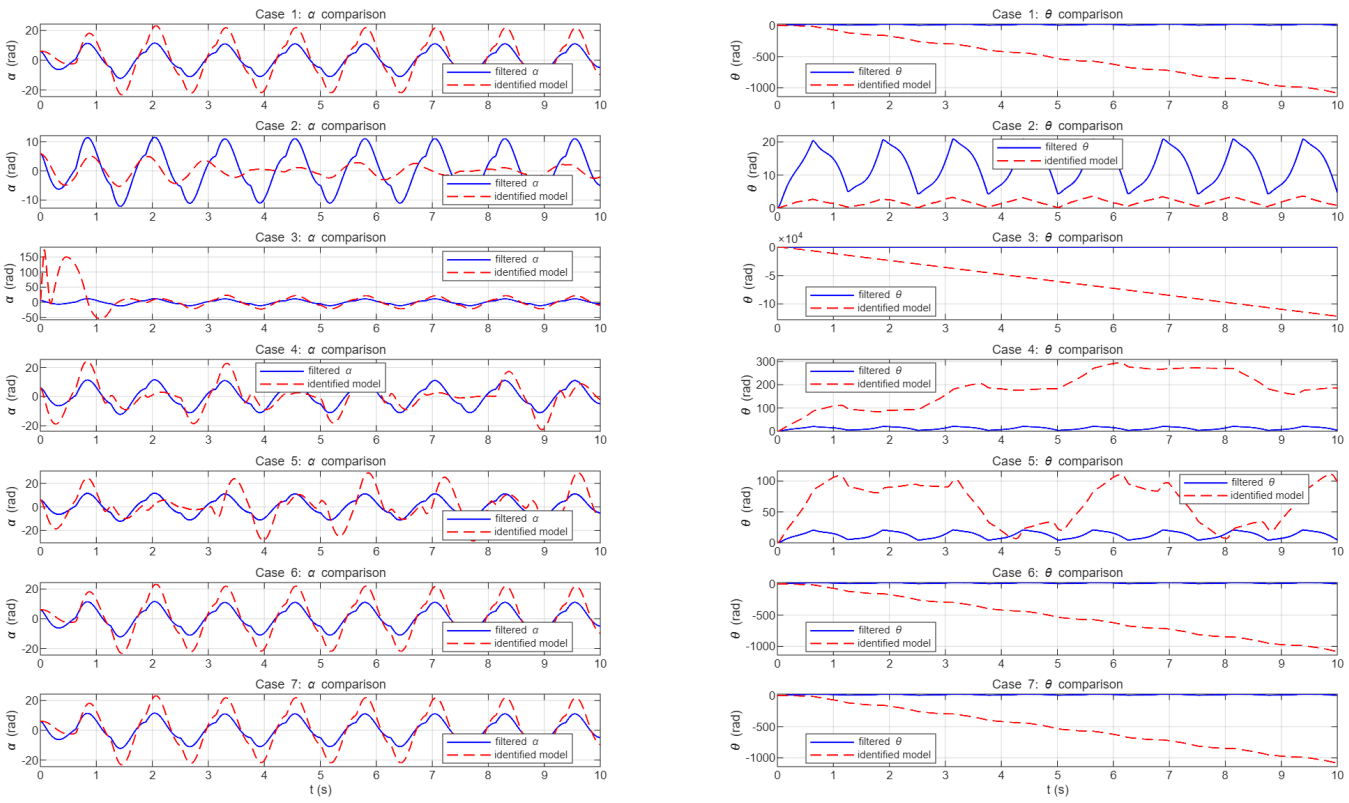
Case	τ_t	K_t	A_c	A_c	K_r	ω_r	ζ_r	RMS$\ddot{\alpha}$
1	0.019922	49.473	-2.7420	-1.0760	-0.9562	6.6684	0.014356	3.1102
2	0.024495	55.468	-2.7672	3.3392	-0.9549	6.6118	0.038892	0.26142
3	0.349400	1715.1	-102.24	-119.38	-0.0823	7.2691	-7.5910	768.54
4	0.020133	49.828	-2.7448	-0.8238	-0.9550	6.6668	0.014390	6.4643
5	0.020150	49.877	-2.7464	-0.7963	-0.9489	6.6584	0.014421	17.181
6	0.019922	49.473	-2.7420	-1.0760	-0.9562	6.6684	0.014356	3.1102
7	0.019922	49.473	-2.7420	-1.0760	-0.9562	6.6684	0.014356	3.1102

Nominal Values	Values
τ_t	0.09695
K_t	1.798
A_c	0.036
A_c	0.400

Nominal Values	Values
K_r	0.02204
ω_r	6.606
ζ_r	0.0378



System Identification Verification — All 7 Cases



Conclusion

Case 1, 2, 3

ω_f must strike a balance between the system's primary frequency and noise. Too low will attenuate the signal, while too high will amplify noise.

Case 1, 4, 5

Noise σ_n directly affects the accuracy of velocity and acceleration derivatives, leading to unstable regression coefficients.

Case 1, 6, 7

Minor input disturbances have little effect on parameter estimation.